Financial Risk & Fraud Analysis SQL Work

This document records the SQL analysis steps completed for the Financial Risk & Fraud Analysis project.

1. Database Selection

Database used: financialriskanalysis

The database contains the financial-risk tables used for joins, aggregation, classification, and analysis.

2. SQL Analysis Workflow

- USE: Select the project database before running the analysis queries.
- JOIN: Combine related tables using matching keys so information from different tables can be analyzed together.
- CASE WHEN: Create categories or labels based on conditions.
- GROUP BY: Aggregate records by a category such as customer type or loan status.
- ORDER BY: Sort the final results to make the highest or lowest values easier to identify.
- VIEW: Save a useful query as a reusable database view.
- Stored Procedure: Create a reusable MySQL procedure that accepts a parameter and returns filtered results.

3. Query: Overdue Loans by Customer Type

Purpose: Count overdue loans for each customer type.

```
USE FinancialRiskAnalysis;

SELECT
   ct.TypeName,
   COUNT(*) AS OverdueLoans
FROM loans l
```

```
JOIN accounts a
    ON l.AccountID = a.AccountID
JOIN customers c
    ON a.CustomerID = c.CustomerID
JOIN customer_types ct
    ON c.CustomerTypeID = ct.CustomerTypeID
JOIN loan_statuses ls
    ON l.LoanStatusID = ls.LoanStatusID
WHERE ls.StatusName = 'Overdue'
GROUP BY ct.TypeName
ORDER BY OverdueLoans DESC;
```

Result shown in the SQL screen: Large Enterprise = 28, Individual = 22, Small Business = 18 overdue loans.

4. Query: Average Loan Amount by Loan Status

Purpose: Calculate the average principal amount for each loan status.

```
USE FinancialRiskAnalysis;

SELECT
    ls.StatusName,
    ROUND(AVG(l.PrincipalAmount), 2) AS AvgLoanAmount
FROM loans l
JOIN loan_statuses ls
    ON l.LoanStatusID = ls.LoanStatusID
GROUP BY ls.StatusName
ORDER BY AvgLoanAmount DESC;
```

Result shown in the SQL screen: Paid Off = 52,664.97; Active = 51,943.77; Overdue = 49,135.81.

5. CASE WHEN Analysis

CASE WHEN was used to classify records according to a condition. For example, loan amounts or balances can be grouped into High, Medium, and Low categories.

```
SELECT
    AccountID,
    Balance,
```

```
    CASE
        WHEN Balance >= 50000 THEN 'High'
        WHEN Balance >= 20000 THEN 'Medium'
        ELSE 'Low'
    END AS BalanceCategory
FROM accounts;
```

6. GROUP BY Analysis

GROUP BY was used with aggregate functions such as AVG() and COUNT() to summarize the data by category.

```
SELECT
    AccountTypeID,
    ROUND(AVG(Balance), 2) AS AverageBalance
FROM accounts
GROUP BY AccountTypeID;
```

7. VIEW

A VIEW was created to save a reusable result that combines account and customer information.

```
CREATE VIEW CustomerAccounts AS
SELECT
    a.AccountID,
    a.CustomerID,
    a.Balance,
    c.FirstName,
    c.LastName
FROM accounts a
JOIN customers c
    ON a.CustomerID = c.CustomerID;
```

8. MySQL Stored Procedure

Because the project uses MySQL Workbench, the stored procedure uses MySQL syntax.

```
DELIMITER //
```

```
CREATE PROCEDURE GetCustomerAccounts(IN p_CustomerID INT)
BEGIN
   SELECT *
   FROM accounts
   WHERE CustomerID = p_CustomerID;
END //

DELIMITER ;
```

9. What Each SQL Step Does

SQL Concept	What It Does
USE	Selects the database.
JOIN	Connects related tables using matching keys.
WHERE	Filters records before aggregation.
CASE WHEN	Creates conditional categories.
COUNT()	Counts records.
AVG()	Calculates an average.
ROUND()	Rounds a numeric result.
GROUP BY	Groups rows for aggregation.
ORDER BY	Sorts the result.
VIEW	Stores a reusable query as a virtual table.
Stored Procedure	Stores reusable SQL logic that can accept parameters.

10. Final Project Workflow

Power Query → Data Cleaning

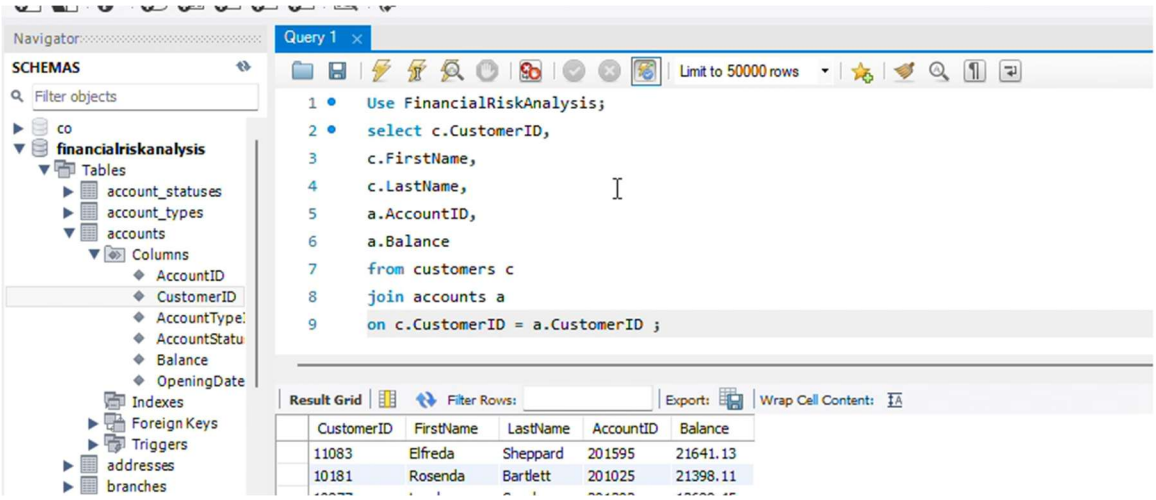
MySQL → SQL Analysis (JOIN, CASE WHEN, GROUP BY, VIEW, Stored Procedure)

Power BI → Data Modeling, DAX, and Dashboard

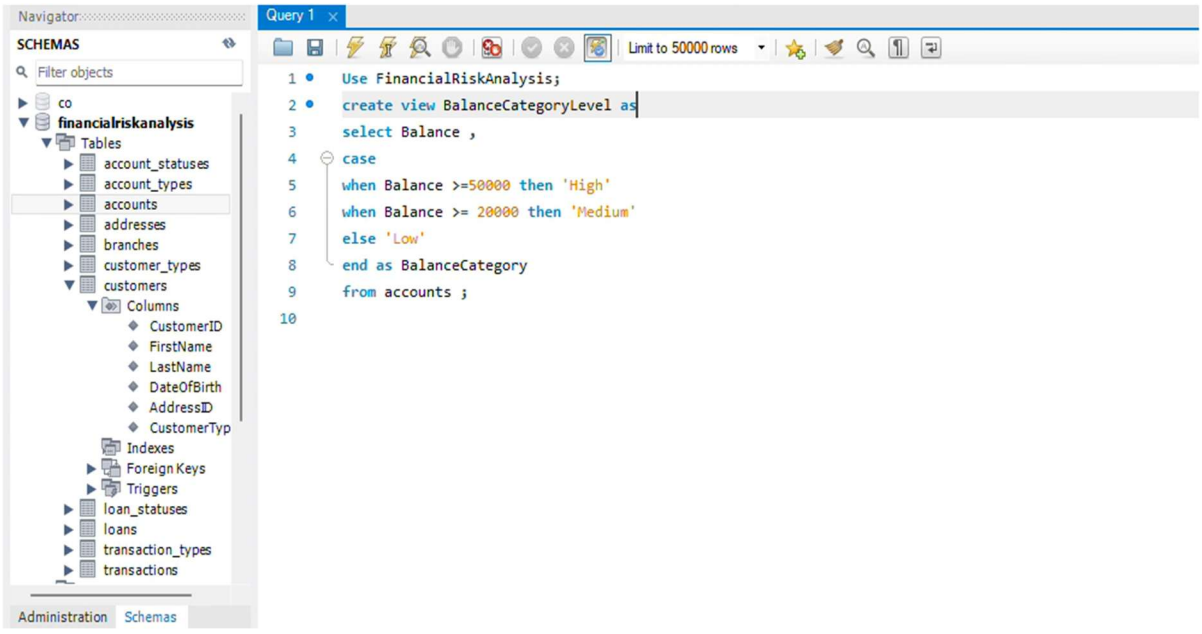
Screenshots from the SQL work are included as evidence of the completed analysis steps.

11. SQL Work Screenshots

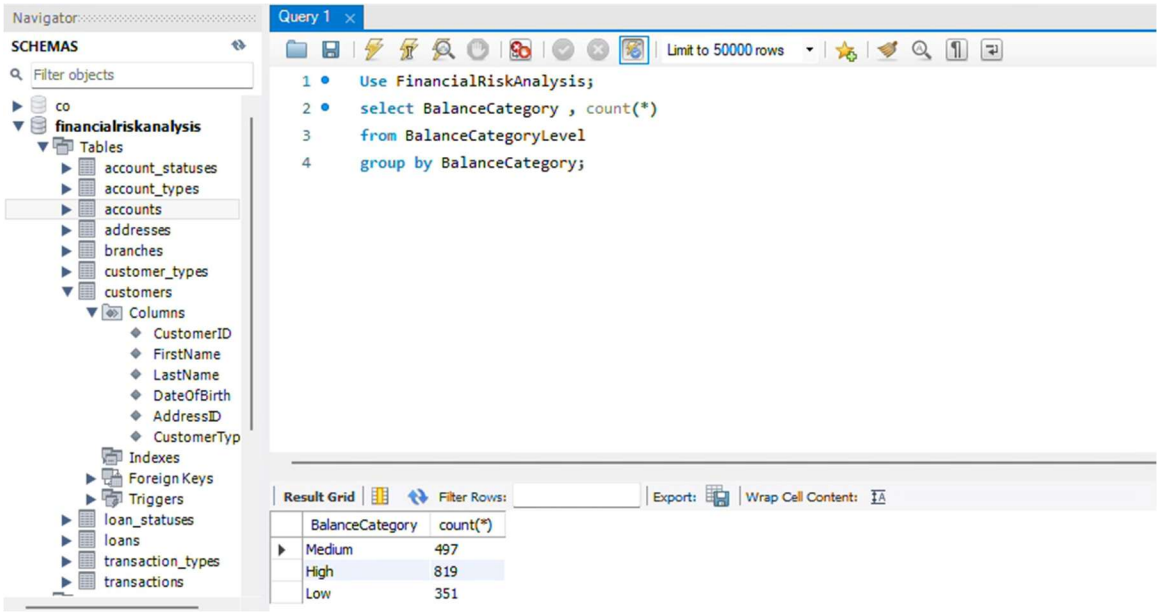
SQL Screenshot 1



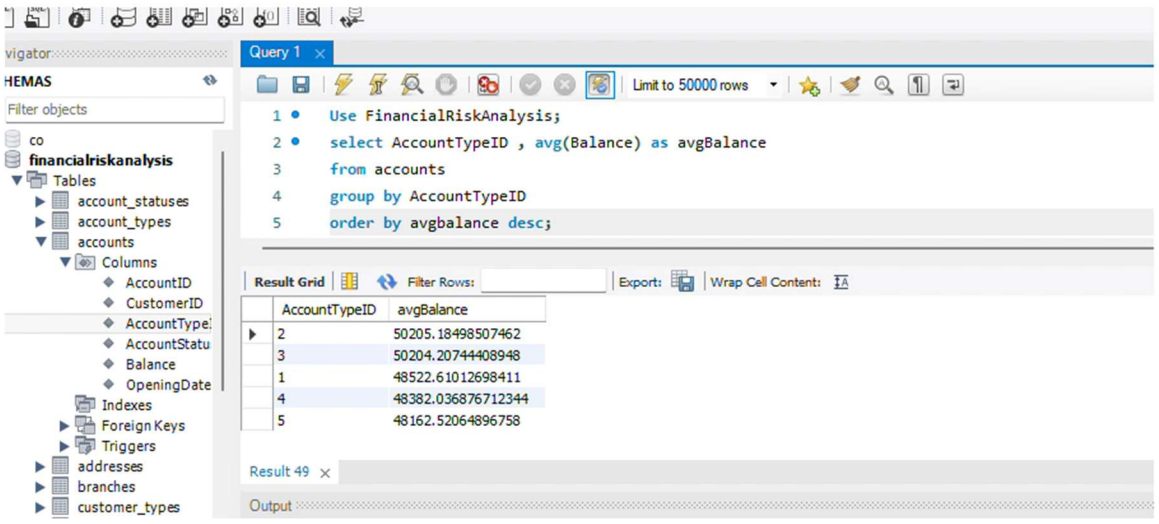
SQL Screenshot 2



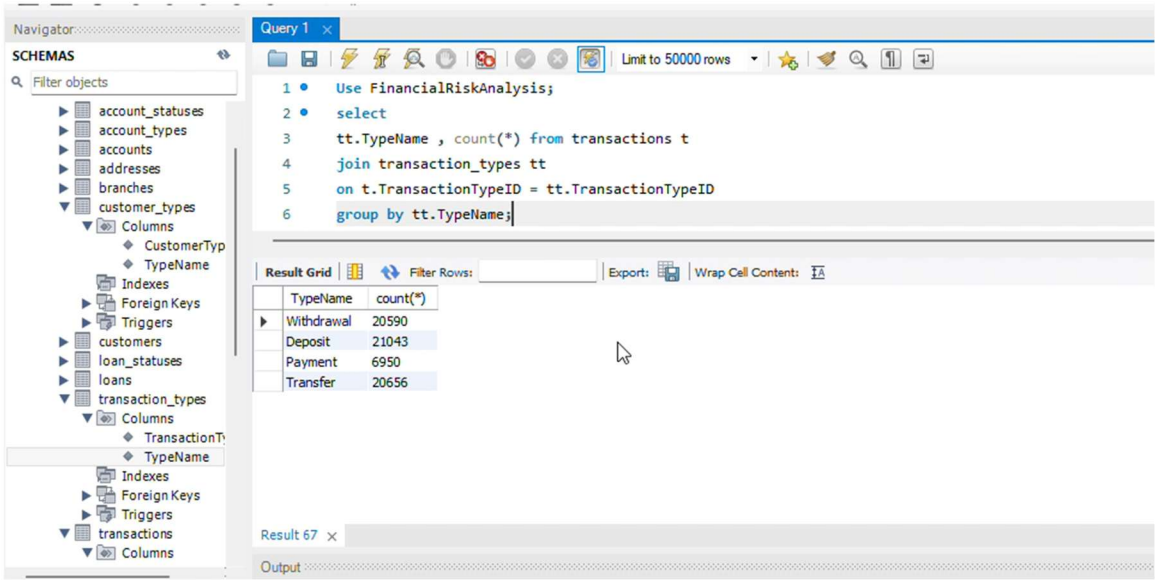
SQL Screenshot 3



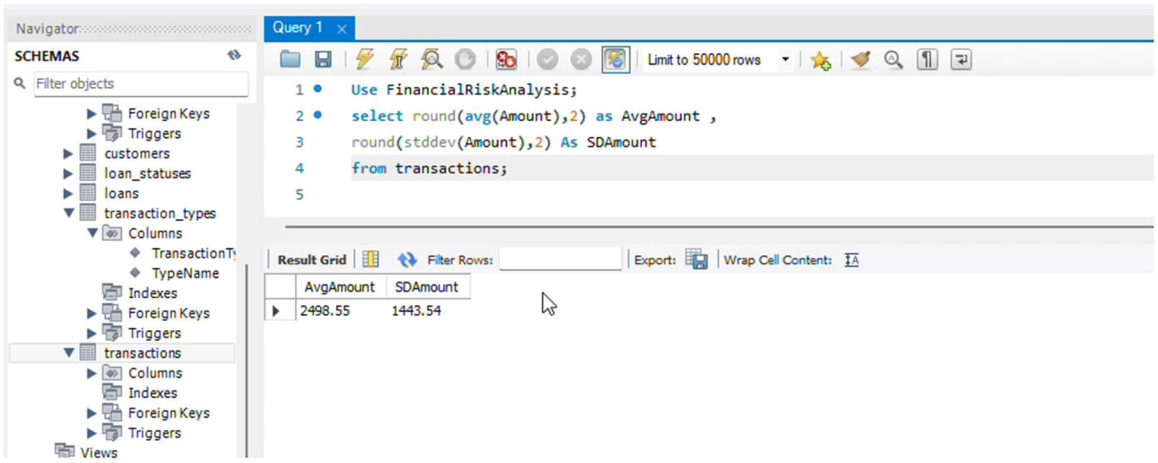
SQL Screenshot 4



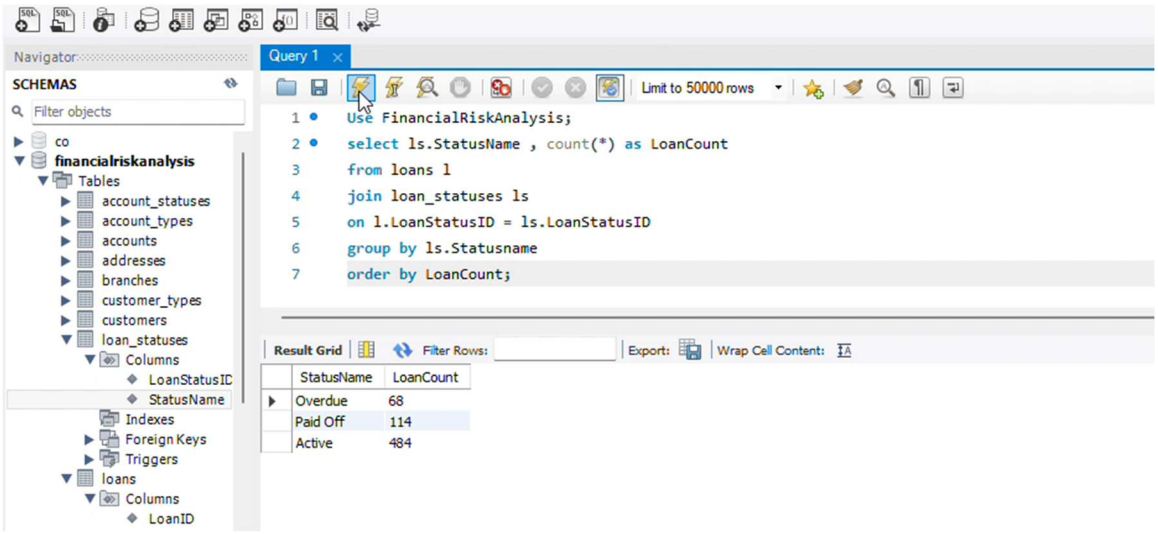
SQL Screenshot 5



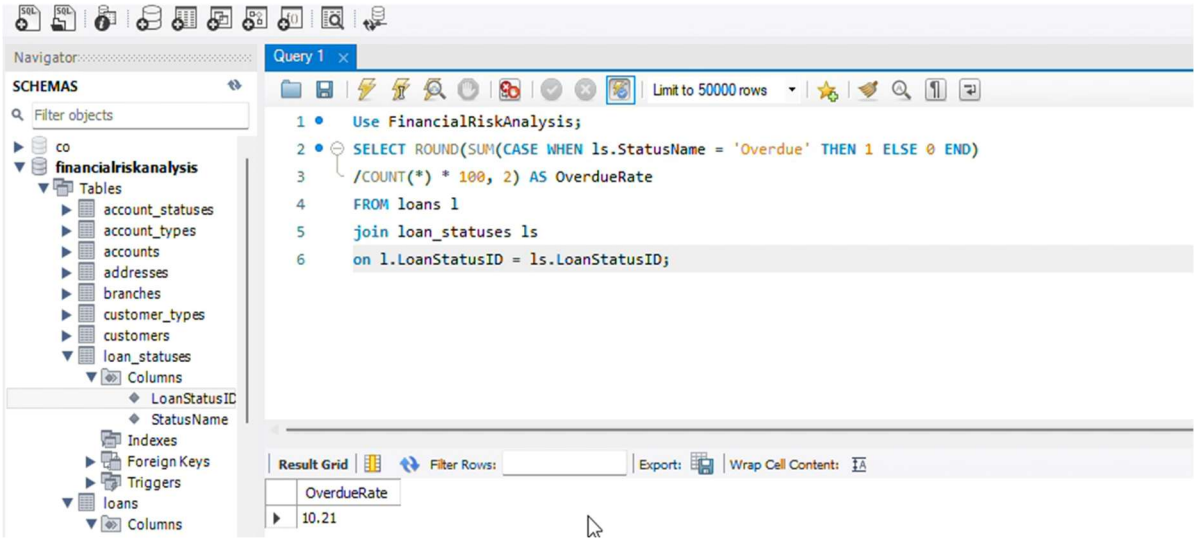
SQL Screenshot 6



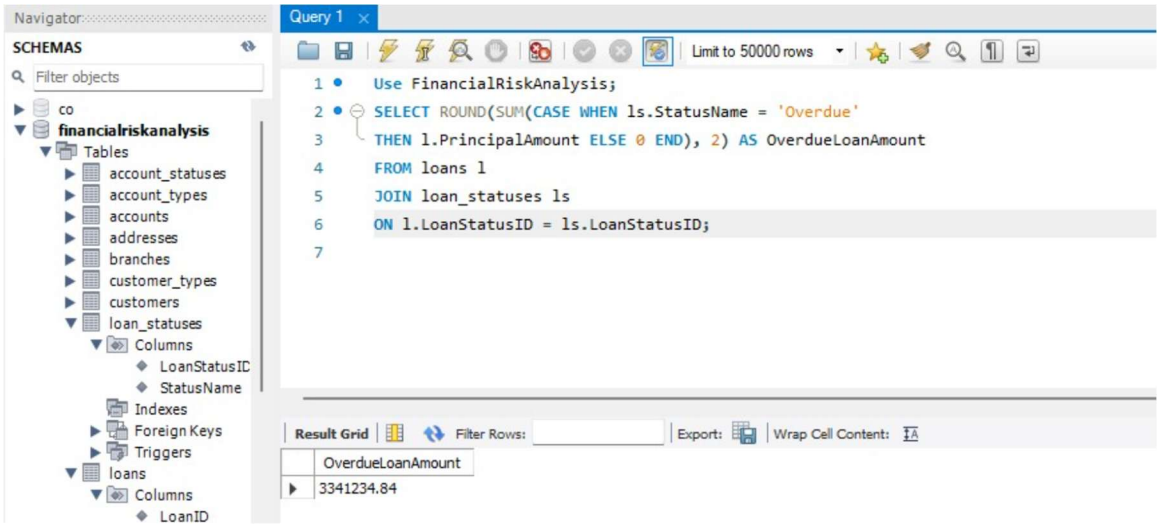
SQL Screenshot 7



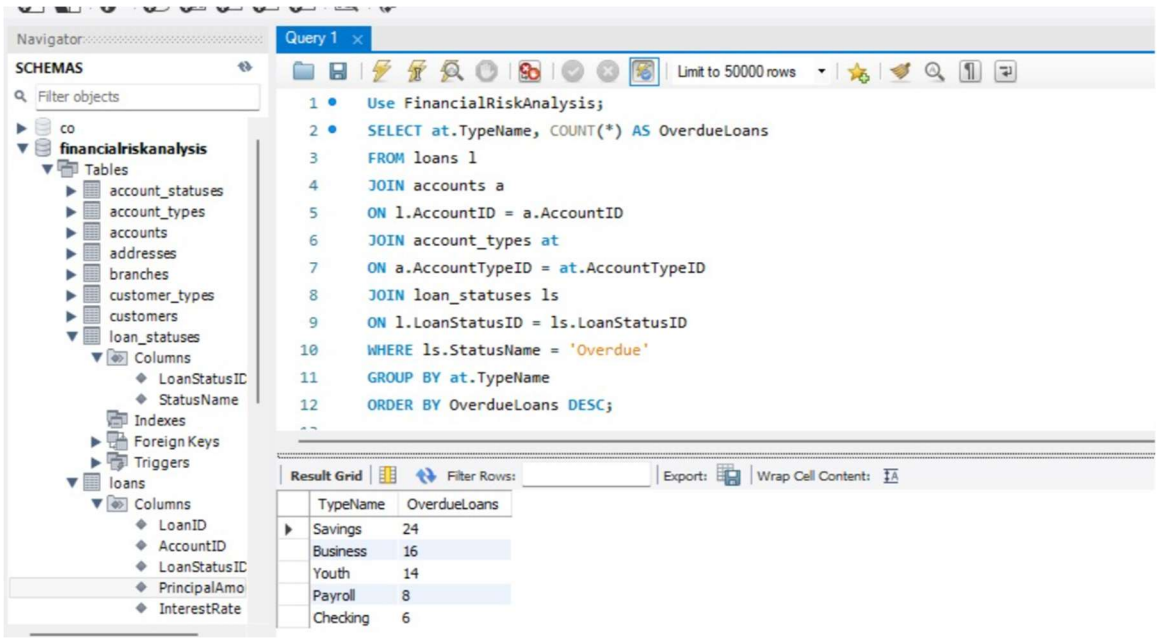
SQL Screenshot 8



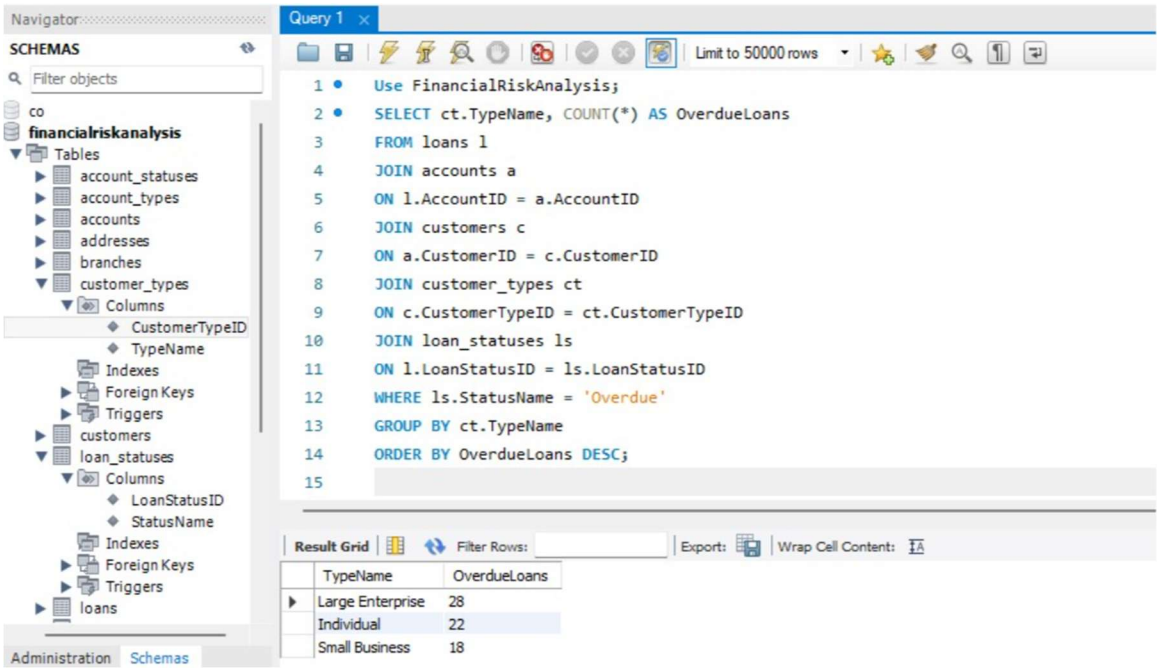
SQL Screenshot 9



SQL Screenshot 10



SQL Screenshot 11



SQL Screenshot 12